

Cómo obtener requisitos de seguridad funcionales y no funcionales

Introducción

Cuando se desarrolla un sistema de software, es común comenzar pensando en las funcionalidades que el usuario necesita: registrar información, iniciar sesión, consultar datos, generar reportes, realizar pagos, administrar usuarios o enviar notificaciones.

Sin embargo, una pregunta fundamental aparece cuando el sistema empieza a manejar información importante:

¿Qué condiciones de seguridad debe cumplir cada una de esas funcionalidades?

Un sistema puede cumplir perfectamente con sus funciones y, aun así, ser inseguro.

Por ejemplo, imaginemos un sistema académico que permite a los profesores consultar las calificaciones de sus estudiantes. Desde el punto de vista funcional, el requisito podría ser:

"El sistema permitirá al profesor consultar las calificaciones de los estudiantes que tiene asignados."

Pero este requisito deja muchas preguntas de seguridad sin responder:

- ¿Puede cualquier profesor consultar cualquier estudiante?
- ¿Puede un estudiante consultar las notas de otro estudiante?
- ¿Puede un profesor modificar las calificaciones?
- ¿Qué ocurre si alguien intenta acceder directamente a una URL protegida?
- ¿Cómo se demuestra quién realizó una modificación?
- ¿Qué ocurre después de varios intentos incorrectos de autenticación?
- ¿Cómo se protegen las calificaciones almacenadas en la base de datos?
- ¿Cuánto tiempo puede permanecer abierta una sesión?
- ¿Qué información debe aparecer en los registros del sistema?

Estas preguntas permiten comprender una idea fundamental:

La seguridad también debe convertirse en requisitos.

La Ingeniería de Requisitos de Seguridad busca precisamente identificar y analizar estas necesidades antes de que el software sea construido. Investigaciones sobre Security Requirements Engineering señalan que los requisitos de seguridad deben identificarse tempranamente en el ciclo de vida del software y que el análisis de amenazas debe formar parte importante de este proceso.

OWASP también plantea que los requisitos de seguridad deben formar parte del proceso de desarrollo y recomienda establecer tanto requisitos funcionales como no funcionales relacionados con seguridad.

1. ¿Qué es un requisito de seguridad?

Un requisito de seguridad expresa una condición que el sistema debe cumplir para proteger información, usuarios, procesos, recursos o servicios.

OWASP define los requisitos de seguridad como declaraciones relacionadas con funcionalidades de seguridad que permiten satisfacer las propiedades de seguridad de una aplicación. Estos requisitos pueden derivarse de estándares de la industria, leyes aplicables y vulnerabilidades conocidas.

En otras palabras, un requisito de seguridad responde a una pregunta sencilla:

¿Qué debe hacer o qué característica debe tener el sistema para evitar que ocurra un problema de seguridad?

Por ejemplo:

"El sistema deberá solicitar autenticación antes de permitir el acceso al módulo de administración."

Este requisito protege el acceso al sistema.

Otro ejemplo:

"El sistema deberá bloquear temporalmente una cuenta después de cinco intentos consecutivos de autenticación fallidos."

Este requisito busca disminuir el riesgo de ataques de fuerza bruta.

Y otro:

"Las contraseñas de los usuarios no deberán almacenarse en texto plano."

Aquí no estamos describiendo una función visible para el usuario, sino una condición de seguridad que debe cumplir la implementación.

2. ¿Por qué no basta con los requisitos funcionales tradicionales?

Supongamos que un cliente solicita:

"El sistema permitirá registrar estudiantes."

Este requisito describe una funcionalidad.

Pero todavía no sabemos:

- quién puede registrar estudiantes;
- qué información puede modificar;
- qué información puede consultar;
- qué ocurre si se intenta registrar información inválida;
- cómo se protege la información personal;
- quién puede eliminar un estudiante;
- cómo queda registrada la eliminación;
- qué ocurre si un usuario no autorizado intenta acceder;
- cómo se controla el acceso al servicio.

Por eso, los requisitos funcionales del negocio deben analizarse desde una perspectiva de seguridad.

Un requisito funcional como:

"El administrador podrá eliminar usuarios."

puede generar requisitos adicionales:

"Solo los usuarios con rol de administrador podrán eliminar usuarios."

"El sistema deberá solicitar confirmación antes de eliminar un usuario."

"El sistema deberá registrar quién realizó la eliminación y cuándo ocurrió."

"El sistema no deberá permitir que un administrador elimine su propia cuenta."

De un solo requisito funcional pueden surgir varios requisitos de seguridad.

Esta es una de las habilidades más importantes que debe desarrollar un ingeniero de software:

aprender a transformar una funcionalidad del negocio en preguntas de seguridad.

3. Requisitos funcionales de seguridad

Los requisitos funcionales describen comportamientos o acciones que el sistema debe realizar.

Cuando hablamos específicamente de requisitos funcionales de seguridad, nos referimos a funcionalidades cuyo propósito está directamente relacionado con la protección del sistema.

Algunos ejemplos son:

- autenticación de usuarios;
- autorización;
- gestión de roles;
- recuperación de contraseñas;
- bloqueo de cuentas;

- cierre de sesión;
- control de acceso;
- registro de eventos de seguridad;
- validación de entradas;
- gestión de sesiones;
- doble factor de autenticación;
- notificaciones ante eventos sospechosos.

Por ejemplo:

"El sistema deberá permitir que los usuarios se autenticuen mediante correo electrónico y contraseña."

Este es un requisito funcional de seguridad.

Otro:

"El sistema deberá permitir asignar los roles Administrador, Profesor y Estudiante."

También es funcional.

Otro:

"El sistema deberá registrar cada intento fallido de autenticación."

También describe un comportamiento que el sistema debe ejecutar.

4. Requisitos no funcionales de seguridad

Los requisitos no funcionales no se concentran principalmente en qué función realiza el sistema, sino en las condiciones, restricciones o características bajo las cuales debe funcionar.

En seguridad pueden estar relacionados con:

- confidencialidad;
- integridad;
- disponibilidad;
- privacidad;
- trazabilidad;
- rendimiento de mecanismos de seguridad;
- resistencia ante ataques;
- protección de datos;
- auditoría;
- cumplimiento normativo.

Por ejemplo:

"Las contraseñas deberán almacenarse utilizando un algoritmo de hash resistente a ataques de fuerza bruta."

No estamos diciendo que el usuario pueda realizar una acción específica. Estamos estableciendo una condición que debe cumplir el sistema.

Otro ejemplo:

"Las sesiones de usuarios deberán expirar después de un período determinado de inactividad."

Otro:

"La información sensible deberá transmitirse mediante un canal cifrado."

Otro:

"Los eventos críticos de seguridad deberán conservar registros suficientes para permitir su auditoría."

Los requisitos no funcionales de seguridad son especialmente importantes porque muchas veces el usuario final no los observa directamente.

5. Una aclaración importante: funcional y no funcional no siempre están completamente separados

Existe una discusión interesante dentro de la Ingeniería de Requisitos de Seguridad.

Algunos investigadores han considerado tradicionalmente los requisitos de seguridad como requisitos no funcionales porque establecen restricciones sobre las funciones del sistema.

Sin embargo, otros trabajos señalan que determinadas necesidades de seguridad deben tratarse como requisitos funcionales porque requieren comportamientos concretos del sistema.

Por ejemplo:

Autenticación

"El usuario deberá autenticarse antes de acceder al sistema."

Esto implica una funcionalidad.

Confidencialidad

"La información sensible deberá mantenerse protegida frente a accesos no autorizados."

Esto expresa una propiedad de seguridad.

Por eso, en la práctica es más útil preguntar:

¿Qué debe hacer el sistema para estar seguro?

y después:

¿Qué condiciones debe cumplir para que esa seguridad sea suficiente?

La primera pregunta ayuda a descubrir requisitos funcionales.

La segunda ayuda a descubrir requisitos no funcionales.

6. El error más común: preguntar únicamente "¿qué seguridad necesita?"

Una de las dificultades para obtener requisitos de seguridad es que los clientes generalmente no hablan utilizando lenguaje técnico.

Un cliente probablemente no dirá:

"Necesitamos implementar control de acceso basado en roles."

Probablemente dirá:

"El profesor no debería poder entrar a las funciones del administrador."

Tampoco necesariamente dirá:

"Necesitamos garantizar la confidencialidad de los datos."

Puede decir:

"No quiero que otros estudiantes puedan ver las notas."

Por eso, el ingeniero de requisitos debe aprender a **convertir necesidades del negocio en requisitos de seguridad**.

7. Primera estrategia: identificar los activos

Una de las mejores formas de comenzar es preguntarse:

¿Qué estamos tratando de proteger?

Estos elementos se conocen como activos.

Un activo puede ser:

- información personal;
- contraseñas;
- información financiera;
- historias clínicas;
- calificaciones;
- documentos;
- fotografías;
- cuentas de usuario;
- información empresarial;
- bases de datos;
- servidores;
- APIs;
- archivos;
- procesos importantes.

Por ejemplo, pensemos nuevamente en un sistema educativo.

Los activos podrían ser:

Activo	¿Por qué debe protegerse?
Datos de estudiantes	Contienen información personal
Calificaciones	No deben ser modificadas por personas no autorizadas
Contraseñas	Permiten acceder a las cuentas
Información de profesores	Puede contener información personal
Reportes académicos	Pueden contener información sensible
Registro de auditoría	Permite conocer quién realizó acciones

Identificar activos ayuda a responder una segunda pregunta:

¿Qué podría ocurrir si este activo es robado, modificado, eliminado o expuesto?

8. Segunda estrategia: identificar las amenazas

Después de identificar los activos debemos preguntarnos:

¿Qué podría salir mal?

Aquí comienza el análisis de amenazas.

Por ejemplo:

Activo: calificaciones

Amenazas:

- un estudiante modifica sus propias calificaciones;
- un profesor modifica calificaciones que no le corresponden;
- alguien accede directamente a una URL;
- un atacante manipula una petición HTTP;
- un usuario roba una sesión;
- alguien obtiene acceso a la base de datos.

Cada amenaza puede conducir a uno o varios requisitos.

Por ejemplo:

Amenaza: estudiante modifica una calificación.

Requisito:

"El sistema deberá impedir que los usuarios con rol Estudiante modifiquen calificaciones."

Amenaza: usuario accede directamente a una URL administrativa.

Requisito:

"El sistema deberá verificar los permisos del usuario antes de permitir el acceso a recursos administrativos."

9. Casos de abuso: pensar como un atacante

Una técnica especialmente útil consiste en dejar de pensar únicamente como usuario y comenzar a pensar como atacante.

Normalmente se construyen casos de uso:

"El administrador registra un usuario."

Pero podemos construir un caso de abuso:

"Un usuario intenta registrar un administrador sin tener permisos."

La pregunta cambia:

¿Cómo podría utilizarse incorrectamente esta funcionalidad?

OWASP recomienda considerar los *Abuse Cases* para identificar posibles ataques y los controles necesarios para mitigarlos; estos análisis pueden alimentar directamente los requisitos de seguridad.

Ejemplo:

Caso normal

Un administrador crea un usuario.

Caso de abuso

Un usuario normal intenta crear un usuario administrador manipulando la petición.

Requisito de seguridad

"El sistema deberá verificar el rol y los permisos del usuario en el servidor antes de permitir la creación o modificación de cuentas administrativas."

Esta forma de pensar es muy poderosa porque obliga a analizar el comportamiento del sistema desde el punto de vista de quien intenta romper las reglas.

10. Tercera estrategia: utilizar preguntas de seguridad

Una entrevista tradicional pregunta:

¿Qué debe hacer el sistema?

Una entrevista orientada a seguridad debe hacer preguntas adicionales.

Por ejemplo:

Sobre usuarios

- ¿Quiénes utilizarán el sistema?
- ¿Qué tipos de usuarios existen?
- ¿Qué puede hacer cada uno?
- ¿Qué información puede consultar cada usuario?
- ¿Qué información puede modificar?
- ¿Existen usuarios privilegiados?

Sobre información

- ¿Qué información es confidencial?
- ¿Qué información puede ser pública?
- ¿Qué información necesita protección especial?
- ¿Qué ocurriría si alguien modifica estos datos?

- ¿Qué ocurriría si los datos desaparecen?

Sobre autenticación

- ¿Cómo deben identificarse los usuarios?
- ¿Qué ocurre después de varios intentos fallidos?
- ¿Se requiere segundo factor?
- ¿Cómo se recupera una contraseña?
- ¿Cuánto tiempo puede permanecer abierta una sesión?

Sobre autorización

- ¿Quién puede consultar?
- ¿Quién puede crear?
- ¿Quién puede modificar?
- ¿Quién puede eliminar?
- ¿Qué ocurre cuando un usuario intenta acceder a algo que no le corresponde?

Sobre auditoría

- ¿Qué acciones deben quedar registradas?
- ¿Quién realizó la acción?
- ¿Cuándo ocurrió?
- ¿Desde dónde se realizó?
- ¿Durante cuánto tiempo deben conservarse los registros?

Este tipo de preguntas permite descubrir requisitos que probablemente el cliente nunca habría mencionado espontáneamente.

11. Cuarta estrategia: utilizar estándares de seguridad

El ingeniero de software no debería intentar inventar todos los requisitos de seguridad desde cero.

Existen estándares y marcos que sirven como fuentes de requisitos.

Uno de los más importantes para aplicaciones web es el **OWASP Application Security Verification Standard (ASVS)**.

OWASP explica que ASVS proporciona requisitos para desarrollar aplicaciones seguras y una base para verificar controles técnicos de seguridad. La versión actual publicada es la 5.0.

ASVS aborda áreas como:

- arquitectura;
- diseño;
- modelado de amenazas;
- autenticación;
- gestión de sesiones;

- control de acceso;
- validación;
- sanitización;
- codificación;
- protección de datos;
- criptografía;
- comunicación;
- API;
- configuración;
- registros y monitoreo.

Por ejemplo, en lugar de preguntarnos únicamente:

"¿Cómo vamos a hacer el login?"

podemos consultar un estándar como ASVS para descubrir qué aspectos de autenticación deben considerarse.

De esta manera, el estándar funciona como una **lista de comprobación para descubrir requisitos que podrían haberse olvidado**.

12. Quinta estrategia: revisar las leyes y regulaciones

Los requisitos de seguridad no provienen únicamente del cliente.

También pueden surgir de:

- legislación;
- regulaciones;
- contratos;
- políticas institucionales;
- estándares de la industria;
- políticas internas;
- requisitos de organismos reguladores.

Por ejemplo, un sistema que almacena información personal puede estar sujeto a obligaciones relacionadas con privacidad y protección de datos.

Por lo tanto, una pregunta importante durante la ingeniería de requisitos es:

¿Qué obligaciones legales o regulatorias debe cumplir el sistema?

Esto es especialmente importante en sistemas:

- bancarios;
- educativos;
- médicos;

- gubernamentales;
 - empresariales;
 - comerciales;
 - financieros.
-

13. Sexta estrategia: realizar modelado de amenazas

El modelado de amenazas permite analizar sistemáticamente cómo podría ser atacado un sistema.

NIST recomienda incorporar principios de ingeniería de seguridad al desarrollo y señala que el modelado de amenazas permite identificar casos de uso, agentes de amenaza, vectores de ataque y controles necesarios para reducir riesgos.

Podemos simplificar el proceso para los estudiantes en cuatro preguntas:

1. ¿Qué tenemos?

Activos.

2. ¿Quién podría atacarlos?

Agentes de amenaza.

3. ¿Cómo podrían atacarlos?

Amenazas o vectores de ataque.

4. ¿Qué debe hacer el sistema para evitarlo?

Requisitos y controles de seguridad.

Por ejemplo:

Activo: cuenta de usuario.

Amenaza: robo de contraseña.

Ataque: credenciales obtenidas mediante phishing.

Requisito posible:

"El sistema deberá permitir autenticación multifactor para cuentas con privilegios administrativos."

14. De una amenaza a un requisito

Esta transformación es fundamental.

Veamos un ejemplo completo.

Funcionalidad

El sistema permitirá a los usuarios iniciar sesión.

Pregunta de seguridad

¿Qué pasa si alguien intenta acceder utilizando una contraseña incorrecta muchas veces?

Amenaza

Ataque de fuerza bruta.

Requisito funcional de seguridad

"El sistema deberá bloquear temporalmente una cuenta después de cinco intentos consecutivos de autenticación fallidos."

Requisito no funcional asociado

"El mecanismo de bloqueo deberá aplicarse de manera consistente a todas las interfaces de autenticación del sistema."

Ahora tenemos algo mucho más concreto.

15. Otro ejemplo: sistema de admisiones escolares

Supongamos que una institución necesita una plataforma de admisiones.

El requisito funcional inicial podría ser:

"El sistema permitirá a los padres registrar la información del aspirante."

Pero inmediatamente debemos formular preguntas:

¿Quién puede consultar esa información?

¿Quién puede modificarla?

¿Qué ocurre si otra persona intenta acceder al formulario?

¿Cómo se protegen los documentos cargados?

¿Qué ocurre con los datos después de finalizar el proceso de admisión?

De estas preguntas pueden surgir requisitos.

Requisitos funcionales de seguridad

El sistema deberá autenticar a los usuarios antes de permitir el acceso a información privada del aspirante.

El sistema deberá permitir que los usuarios consulten únicamente los procesos de admisión que les correspondan.

El sistema deberá registrar las modificaciones realizadas sobre la información del aspirante.

El sistema deberá impedir que un usuario elimine documentos sin los permisos correspondientes.

Requisitos no funcionales

La información personal de los aspirantes deberá transmitirse utilizando mecanismos de comunicación segura.

Los documentos almacenados deberán contar con mecanismos de protección contra acceso no autorizado.

Las contraseñas deberán almacenarse mediante mecanismos criptográficos apropiados.

Los registros de auditoría deberán conservar información suficiente para determinar quién realizó una operación y cuándo.

Aquí podemos observar que la seguridad no constituye una única funcionalidad.

Está distribuida por todo el sistema.

16. La importancia de escribir requisitos verificables

Un requisito de seguridad debe poder comprobarse.

Comparemos:

Requisito débil

"El sistema debe ser muy seguro."

Este requisito no puede probarse fácilmente.

¿Qué significa "muy seguro"?

Requisito mejor definido

"El sistema deberá bloquear temporalmente la autenticación de una cuenta después de cinco intentos consecutivos fallidos."

Ahora sí podemos diseñar una prueba.

Intento 1 → acceso rechazado.

Intento 2 → acceso rechazado.

Intento 3 → acceso rechazado.

Intento 4 → acceso rechazado.

Intento 5 → cuenta bloqueada.

Esto permite convertir el requisito en un criterio de aceptación.

17. Evitar palabras ambiguas

En requisitos de seguridad debemos tener especial cuidado con palabras como:

- rápido;
- seguro;
- suficiente;
- adecuado;
- protegido;
- confiable;
- apropiado;
- periódicamente.

Estas palabras pueden tener interpretaciones diferentes.

Por ejemplo:

"El sistema deberá realizar copias de seguridad periódicamente."

¿Qué significa periódicamente?

¿Cada hora?

¿Cada día?

¿Cada semana?

Un requisito mejor definido sería:

"El sistema deberá realizar una copia de seguridad automática de la base de datos cada 24 horas."

Ahora podemos verificarlo.

18. Seguridad desde la perspectiva de la tríada CIA

Una forma sencilla de analizar requisitos de seguridad es utilizar la tríada:

Confidencialidad – Integridad – Disponibilidad.

Confidencialidad

Busca impedir que personas no autorizadas conozcan información.

Pregunta:

¿Quién puede ver esta información?

Ejemplo:

"Los estudiantes no podrán consultar las calificaciones de otros estudiantes."

Integridad

Busca impedir modificaciones no autorizadas o incorrectas.

Pregunta:

¿Quién puede modificar esta información?

Ejemplo:

"Solo los profesores asignados al curso podrán modificar las calificaciones."

Disponibilidad

Busca que el sistema o información esté disponible cuando sea necesario.

Pregunta:

¿Qué ocurre si el sistema deja de funcionar?

Ejemplo:

"El sistema deberá contar con mecanismos de recuperación que permitan restaurar la información después de una falla."

19. Requisitos de seguridad y casos de uso

Otra técnica consiste en revisar cada caso de uso y preguntarse:

¿Qué puede salir mal?

Por ejemplo:

Caso de uso

"Registrar usuario."

Preguntas de seguridad

- ¿Quién puede realizarlo?
- ¿Qué información puede registrar?
- ¿Puede registrar otro administrador?
- ¿Qué pasa si introduce datos maliciosos?
- ¿Qué pasa si intenta registrar un usuario que ya existe?
- ¿Debe quedar registro de la operación?
- ¿Puede modificar posteriormente la información?

Cada respuesta puede generar un requisito.

Así, la seguridad deja de ser un documento separado y comienza a integrarse directamente en los casos de uso.

20. Requisitos derivados de los roles

Una de las fuentes más importantes de requisitos de seguridad son los roles.

Supongamos:

- Administrador
- Profesor
- Estudiante

Podemos construir una matriz:

Funcionalidad	Administrador	Profesor	Estudiante
Crear usuarios	Sí	No	No
Consultar estudiantes	Sí	Sí	No
Modificar notas	Sí	Sí	No
Consultar notas	Sí	Sí	Sí
Eliminar usuarios	Sí	No	No

Esta tabla permite descubrir requisitos de autorización.

Por ejemplo:

"El sistema deberá impedir que los estudiantes ejecuten operaciones de modificación de calificaciones."

Y aquí aparece una enseñanza importante:

Ocultar un botón no es lo mismo que controlar el acceso.

Si el botón "Eliminar usuario" no aparece para un estudiante, eso es una característica de interfaz.

Pero el requisito de seguridad real es:

"El servidor deberá rechazar cualquier solicitud de eliminación de usuarios realizada por un usuario que no posea los permisos correspondientes."

21. ¿De dónde pueden salir los requisitos de seguridad?

Un buen proceso de obtención de requisitos no utiliza una única fuente.

Los requisitos pueden surgir de:

1. Usuarios

Entrevistas y conversaciones.

2. Cliente

Necesidades del negocio.

3. Casos de uso

Análisis de funcionalidades.

4. Casos de abuso

Análisis de comportamientos maliciosos.

5. Amenazas

Modelado de amenazas.

6. Vulnerabilidades conocidas

Experiencias de incidentes anteriores.

7. Estándares

Por ejemplo, OWASP ASVS.

8. Legislación

Protección de datos y otras obligaciones.

9. Arquitectura

Decisiones relacionadas con infraestructura, APIs, bases de datos y comunicaciones.

10. Pruebas

Los resultados de pruebas de seguridad pueden revelar nuevos requisitos.

La literatura sobre captura de requisitos de seguridad también propone utilizar conocimiento previo y catálogos de seguridad para ayudar a descubrir requisitos de manera sistemática.

22. Una metodología sencilla para los estudiantes

Para un proyecto de aula podemos utilizar el siguiente procedimiento:

Paso 1. Identificar funcionalidades

¿Qué hará el sistema?

Ejemplo:

Registrar estudiantes.

Paso 2. Identificar usuarios

¿Quién utilizará esa funcionalidad?

Ejemplo:

Administrador y profesor.

Paso 3. Identificar activos

¿Qué información se está manejando?

Ejemplo:

Datos personales del estudiante.

Paso 4. Identificar amenazas

¿Qué podría salir mal?

Ejemplo:

Un usuario no autorizado accede a la información.

Paso 5. Identificar controles

¿Qué podría hacer el sistema para evitarlo?

Ejemplo:

Control de acceso por roles.

Paso 6. Formular el requisito

"El sistema deberá permitir consultar información de estudiantes únicamente a usuarios autorizados."

Paso 7. Determinar si es funcional o no funcional

¿Describe una acción que el sistema realiza?

→ Funcional.

¿Describe una condición o propiedad que debe cumplir?

→ No funcional.

Paso 8. Crear el criterio de aceptación

¿Cómo comprobaremos que funciona?

Ejemplo:

"Cuando un estudiante intente acceder a la información privada de otro estudiante, el sistema deberá rechazar la solicitud."

23. Una plantilla para escribir requisitos de seguridad

Una estructura sencilla puede ser:

ID: RS-001

Tipo: Funcional

Requisito:

El sistema deberá permitir únicamente a los usuarios autenticados acceder al módulo de administración.

Justificación:

Evitar accesos no autorizados.

Amenaza asociada:

Acceso de usuarios no autenticados.

Criterio de aceptación:

Un usuario sin autenticación que intente acceder al módulo deberá ser redirigido al mecanismo de autenticación.

Esta estructura ayuda a relacionar:

amenaza → requisito → implementación → prueba.

24. Ejemplo de catálogo de requisitos

ID	Tipo	Requisito	Amenaza
RS-001	Funcional	El sistema deberá autenticar a los usuarios antes de permitir el acceso a recursos protegidos.	Acceso no autorizado
RS-002	Funcional	El sistema deberá controlar el acceso según el rol del usuario.	Escalamiento de privilegios
RS-003	Funcional	El sistema deberá registrar los intentos fallidos de autenticación.	Ataques de fuerza bruta
RS-004	Funcional	El sistema deberá cerrar la sesión cuando el usuario solicite finalizarla.	Uso indebido de sesiones
RS-005	No funcional	Las contraseñas no deberán almacenarse en texto plano.	Robo de credenciales
RS-006	No funcional	La comunicación entre cliente y servidor deberá utilizar un canal seguro.	Interceptación
RS-007	No funcional	Los registros de auditoría deberán permitir identificar al usuario que realizó una operación.	Falta de trazabilidad
RS-008	No funcional	El sistema deberá mantener la confidencialidad de los datos sensibles.	Exposición de información

25. ¿Y dónde entra Spring Security?

Aquí aparece la conexión con el curso de Desarrollo de Software II.

Spring Security no debería ser el punto de partida para descubrir los requisitos.

Primero se determina:

¿Qué necesita proteger el sistema?

Después:

¿Qué requisitos de seguridad necesita?

Y finalmente:

¿Qué tecnología utilizaremos para implementarlos?

Por ejemplo:

Necesidad

Los usuarios deben identificarse.

Requisito

El sistema deberá autenticar a los usuarios antes de permitir el acceso a recursos protegidos.

Decisión técnica

Spring Security.

Otro ejemplo:

Necesidad

Los estudiantes y administradores tienen permisos diferentes.

Requisito

El sistema deberá controlar el acceso a los recursos según los permisos asignados al usuario.

Decisión técnica

Spring Security + roles/authorities.

Esto es importante porque evita enseñar seguridad como una colección de configuraciones.

El estudiante debe comprender que:

Spring Security implementa decisiones de seguridad que primero debieron ser identificadas como requisitos.

26. Seguridad como parte del ciclo de vida

La seguridad no debería aparecer únicamente cuando comienza la programación.

Una investigación sobre Ingeniería de Requisitos de Seguridad señala precisamente la importancia de identificar los requisitos de seguridad desde las primeras etapas del ciclo de vida, evitando descubrir problemas demasiado tarde.

Un proceso simplificado sería:

Necesidad del negocio

↓

Requisitos funcionales

↓

Activos

↓

Amenazas

↓

Requisitos de seguridad

↓

Diseño

↓

Implementación

↓

Pruebas

↓

Validación

Si esperamos hasta las pruebas para preguntarnos si el sistema es seguro, posiblemente ya sea demasiado tarde.

27. Los requisitos de seguridad también pueden cambiar

Otro aspecto importante es que la seguridad no termina cuando se publica el sistema.

Las amenazas cambian.

Aparecen:

- nuevas vulnerabilidades;
- nuevos ataques;

- nuevas tecnologías;
- nuevos requisitos legales;
- nuevos tipos de usuarios;
- nuevos servicios;
- nuevas integraciones.

Por eso, los requisitos de seguridad también deben revisarse durante la evolución del sistema.

Investigaciones sobre evolución de sistemas señalan que pueden aparecer nuevos requisitos de seguridad durante otras etapas del ciclo de vida debido al cambio constante del panorama de amenazas.

28. Una pregunta fundamental para el futuro ingeniero de software

Cuando un cliente diga:

"Necesito un sistema para gestionar estudiantes."

El desarrollador no debería comenzar inmediatamente a crear clases, controladores, entidades o pantallas.

Debe comenzar preguntando:

¿Qué información vamos a manejar?

¿Quién utilizará el sistema?

¿Qué puede hacer cada usuario?

¿Qué información necesita protegerse?

¿Qué podría salir mal?

¿Quién podría intentar aprovecharse de una vulnerabilidad?

¿Qué consecuencias tendría un ataque?

¿Qué requisitos de seguridad debemos establecer?

¿Cómo comprobaremos que esos requisitos se cumplen?

Estas preguntas permiten pasar de:

"Necesitamos que el sistema sea seguro."

a:

"El sistema deberá impedir que un usuario sin autorización consulte información perteneciente a otro usuario."

La segunda afirmación es mucho más útil porque puede diseñarse, implementarse y probarse.

29. Conclusión

Obtener requisitos de seguridad no consiste simplemente en preguntarle al cliente:

"¿Qué seguridad necesita?"

La seguridad debe descubrirse mediante un proceso sistemático.

Primero se identifican las funcionalidades del sistema. Después se identifican los usuarios, activos y datos importantes. Luego se analizan amenazas y posibles casos de abuso. A partir de ese análisis se construyen requisitos de seguridad funcionales y no funcionales.

También es importante utilizar fuentes externas como estándares, legislación, vulnerabilidades conocidas y buenas prácticas de la industria. OWASP recomienda precisamente establecer requisitos técnicos de seguridad funcionales y no funcionales y utilizar ASVS como guía para definirlos.

El **OWASP ASVS** resulta especialmente útil porque transforma la seguridad en requisitos concretos que pueden utilizarse durante el diseño, desarrollo y pruebas de una aplicación.

Finalmente, los requisitos deben ser claros, específicos, verificables y relacionados con amenazas reales.

La idea central que debe quedar en el estudiante es:

La seguridad no se agrega al software después de programarlo. La seguridad se descubre, se especifica, se diseña, se implementa y se verifica desde los requisitos.

Y existe una segunda idea todavía más importante:

Un buen desarrollador no solamente pregunta qué debe hacer el sistema; también pregunta qué podría salir mal cuando alguien intente utilizarlo de manera incorrecta o maliciosa.

Fuentes recomendadas para profundizar

- **OWASP Application Security Verification Standard (ASVS)** — estándar de referencia para definir y verificar requisitos de seguridad en aplicaciones web.
- **OWASP Developer Guide – Requirements** — explica cómo incorporar requisitos de seguridad al desarrollo y cómo utilizar casos de abuso para descubrirlos.
- **OWASP – Establishing a Modern Application Security Program** — aborda específicamente la recopilación de requisitos de negocio y la definición de requisitos funcionales y no funcionales de seguridad.

- **NIST IR 8397** — presenta técnicas de verificación de software como modelado de amenazas, pruebas automatizadas, análisis estático, fuzzing y escaneo de aplicaciones web.
- **Survey and analysis on Security Requirements Engineering** — investigación académica sobre métodos y actividades para obtener requisitos de seguridad.
- **Capturing security requirements for software systems** — propuesta de metodología para obtener requisitos de seguridad de forma sistemática.